

FLUTTER SESLİ SOHBET ODASI — TAM UYGULAMA PROMPTU

Amaç: Flutter uygulamasındaki Sesli Sohbet sistemini, web sitesiyle **%100 aynı** davranacak şekilde yeniden düzenlemek.

Kurallar (DEĞİŞTİRİLEMEZ):

- ❌ Yeni backend / yeni API / yeni endpoint **YAZILMAYACAK**.
- ❌ Mock veri **KULLANILMAYACAK**.
- ✅ Web sitesinin kullandığı **MEVCUT** API, SSE, veritabanı ve iş mantığı kullanılacak.
- ✅ Tüm değişiklikler web ile **senkron** çalışacak.
- ✅ Acceptance testlerinde **0 FAIL / 0 WARNING** olmadan APK/AAB üretilmeyecek.

Base URL: `https://canlifal.com`

Auth: Tüm istekler `Authorization: Bearer <accessToken>` header'ı ile yapılır (mobil JWT). Backend tüm endpoint'lerde `authenticateRequest()` ile dual-auth (mobil JWT veya web session) destekler.

0. GENEL MİMARİ (BACKEND GERÇEĞİ)

Sesli oda durumu **tek kaynaktan (SSE stream)** akar. Flutter asla kendi state'ini uydurmaz; her şey backend'den gelir.

0.1 SSE Bağlantısı (TEK KANAL — her şey buradan gelir)

```
GET /api/chat/rooms/{roomId}/stream
Header: Authorization: Bearer <accessToken>
Content-Type: text/event-stream
```

Backend 2 saniyede bir poll eder ve şu **event tiplerini** gönderir (her biri `data: {json}\n\n` formatında):

type	İçerik	Kullanım
connected	{ type, roomId }	Bağlantı kuruldu
messages	{ type, messages: ChatMessage[] }	Yeni sohbet mesajları
system	{ type, event, ... }	Moderasyon olayları (aşağıda)
gift	{ type, ...giftData }	Hediye animasyonu
presence	{ type, users: PresenceUser[] }	Odadaki kullanıcılar (10 sn'de bir)
dj	{ type: 'dj', event: 'QUEUE_UPDATED', playing, nowPlaying, musicUrl, musicQueue, queueLength }	Müzik durumu (ilk poll'da tam state, sonra değişimde)
typing	{ type, users: string[] }	Yazıyor göstergesi

system event'leri (event alanı):

USER_MUTED , USER_UNMUTED , USER_KICKED , USER_BANNED , ROOM_MUTED , ROOM_UNMUTED , CHAT_CLEARED , ANNOUNCEMENT .

KRİTİK: Flutter, müzik/queue/presence için ayrı polling yapmaz. SSE dj event'i tek doğruluk kaynağıdır. Sadece SSE koparsa fallback REST çağırısı yapılır.

0.2 Reconnect Mantığı

- SSE koptuğunda exponential backoff (1s → 2s → 4s → ... max 30s, jitter ekle).
- Yeniden bağlanınca backend ilk poll'da **tam dj payload** + presence gönderir → state otomatik resync olur.
- App resume olunca (foreground) bağlantı yeniden kurulur ve nowPlaying.startedAt 'a göre müzik **kaldığı saniyeden** devam eder (bkz. §6).

1. ALT MENÜ — GEREKSİZ BUTONLARI KALDIR

Sesli sohbet odası alt menüsünden **tamamen kaldır**:

- **✗ Müzik Aç** butonu (müzik artık §17 popup + !istek komutu ile çalışır)
- **✗ DJ** butonu
- **✗ PK** butonu
- **✗ Sağ taraftaki ok (>) işareti**

Başka gereksiz hiçbir buton kalmayacak. Alt menüde sadece: **mesaj yazma alanı, mikrofon, hediye, çark (komutlar §11), arkaplan (§10)** kalır.

2. MÜZİK SİSTEMİ — WEB İLE BİREBİR

2.1 İstek Yöntemleri

1. **Komut:** Sohbet !istek Şarkıcı - Şarkı yazılır.
2. **Popup:** \$17'deki arama popup'ından seçilir.

Her ikisi de aynı backend akışını tetikler (aşağıda).

2.2 Şarkı İsteği Gönderme (TEK ENDPOINT)

```
POST /api/chat/rooms/{roomId}/song-request
Body: {
  videoId: string,           // YouTube video ID (zorunlu)
  title: string,             // şarkı başlığı (zorunlu)
  duration?: string,         // "3:45" formatı (6 dk kontrolü için ÖNEMLİ)
  requestType: 'audio' | 'video', // audio=10 jeton, video=20 jeton
  dedication?: string,       // ithaf
  note?: string              // not
}
```

Backend davranışı (aynen uygula, Flutter tarafında tekrar etme):

- Jeton kontrolü ve düşümü backend yapar (audio 10 / video 20). Yetersizse `400 { error: "Yetersiz jeton. X jeton gerekiyor." }`.
- 6 dakika kontrolü backend yapar → `400 { error: "Şarkı 6 dakikadan uzun olamaz..." }`.
- **Oda boşsa** (`currentMusicVideoId` yok) → hemen çalmaya başlar (`startedImmediately: true`).
- **Müzik çalışırsa** → otomatik **kuyruğa** eklenir (`queued: true` , `queuePosition: N`). **Mevcut müzik YENİDEN BAŞLAMAZ, KAPANMAZ.**
- Cevap:


```
{ success, newBalance, queued, startedImmediately, queuePosition, playing, nowPlaying, musicUrl, queue, musicQueue, queueLength }
```
- Backend `emitDjUpdate()` çağırır → SSE `dj` event'i tüm odaya gider. **Flutter sadece SSE'yi dinler, cevabı UI güncellemek için kullanmaz (çift güncelleme önlenir).**

2.3 Çözülecek Hatalar (ZORUNLU DAVRANIŞ)

Hata	Çözüm
Müzik çok geç bulunuyor	Arama debounce 250ms + ön bellek (§17/§18). Stream URL çözümü <code>youtube-stream</code> cache'li (60sn).
Müzik geç başlıyor	İstek gönderilir gönderilmez SSE <code>dj event</code> 'i gelir; player önceden hazır (tek <code>AudioPlayer / WebView instance</code>) tutulur, yeniden oluşturulmaz.
Yeni istek gelince mevcut müzik yeniden başlıyor	YASAK. Müzik çalışıyorsa istek kuyruğa gider; player'a dokunulmaz . SSE <code>dj event</code> 'inde <code>nowPlaying.videoId</code> değişmediyse player'ı resetleme (<code>videoId</code> karşılaştır).
Çalan müzik kapanıyor	<code>nowPlaying.videoId</code> aynıysa player'ı yeniden kurma; sadece queue UI güncelle.
Yeni istek sıraya eklenmiyor	<code>musicQueue</code> SSE'den gelir, listeyi direkt ondan render et.
Sıradaki liste çalışmıyor	<code>dj.musicQueue</code> array'ini sırayla göster (paralı önce, sonra kronolojik — backend zaten sıralı gönderir).
Sıradan silme çalışmıyor	Aşağıdaki DELETE-eşdeğeri için: kuyruktaki öge <code>id</code> ile PATCH/işaretleme yapılır (bkz §2.5).
Müzik durunca sıradaki otomatik başlamıyor	DELETE <code>/music</code> çağrısı backend'de <code>playNextFromQueue()</code> ile otomatik sonrakine geçer.
Kuyruk backend ile senkron değil	Asla local queue tutma; tek kaynak <code>dj.musicQueue</code> .

2.4 Çalan Müziği Durdurma / Sonrakine Geçme

DELETE `/api/chat/rooms/{roomId}/music`  mevcut müziği durdurur, kuyrukta varsa otomatik sonrakini başlatır (autoAdvanced)

Sadece yetkili (oda sahibi / admin / aktif DJ) çağırabilir — backend `canControlMusic()` ile kontrol eder, yetkisizse 403.

2.5 Kuyruktan Şarkı Silme / Atlama

Kuyruktaki bir şarkıyı **çalınmış işaretleyerek atlamak** (sonrakine geçirmek):

```
PATCH /api/chat/rooms/{roomId}/song-request
Body: { requestId: <queue item id> }
```

Bu öğeyi [PLAYED] işaretler ve seçilen şarkıyı current music yapar; emitDjUpdate() ile SSE gider.

2.6 Mevcut Müzik / Kuyruk Okuma (fallback / ilk açılış)

```
GET /api/chat/rooms/{roomId}/music → { videoId, title, startedAt, duration, requestType, playing, nowPlaying, musicUrl, musicQueue }
GET /api/chat/rooms/{roomId}/music-queue → { queue, total, playing, nowPlaying, musicUrl, musicQueue, queueLength }
GET /api/chat/rooms/{roomId}/song-request → { queue, playing, nowPlaying, musicUrl, musicQueue, requestCosts:{audio:10,video:20} }
```

Normalde SSE yeterli; bu GET'ler yalnız ilk ekran açılışında veya reconnect sonrası tek seferlik resync için.

3. JETON SİSTEMİ — SES / VIDEO SEÇİMİ

Müzik gönderirken kullanıcıya **seçim popup'**ı çıkar:

- 🎧 **Sadece Ses** → requestType: 'audio' → **10 Jeton**
- 🎬 **Videolu** → requestType: 'video' → **20 Jeton**

Fiyatlar **backend'den** gelir: GET /api/chat/rooms/{roomId}/song-request cevabındaki requestCosts: { audio: 10, video: 20 }. **Sabit kodlama yapma**, bu değeri kullan. Jeton düşümünü backend yapar; Flutter sadece cevaptaki newBalance ile bakiyeyi günceller.

4. SÜRE LİMİTİ — 6 DAKİKA

- 6 dakikadan (360 sn) uzun hiçbir video/müzik **tamamen** oynatılmaz. Ses ve videolu mod için **geçerli**.
- Backend istek anında reddeder: duration > 360 ise 400 { error: "Şarkı 6 dakikadan uzun olamaz..." } → Flutter bu hatayı kullanıcıya gösterir.
- Süresi bilinmeyen/uzun şarkılar için: player tarafında **hard cap 360 sn** uygula → 6. dakikada otomatik DELETE /music çağır (sonrakine geç). Backend de GET /music sırasında elapsed > duration+5 olursa otomatik temizler (DEFAULT_MAX_DURATION_SECONDS = 360).

5. VIDEO PLAYER — FLOATING (ORTADA)

Videolu müzik (requestType:'video' veya nowPlaying ait olduğu istek VIDEO):

- Oda ekranının **tam ortasında floating player** olarak görünür.
- **Chat yazarken kapanmaz** (klavye açılınca player kalır, üstte/ortada sabit).
- **Oda değiştirilmediği sürece** görünmeye devam eder (sayfa state'i ile bağlı, her rebuild'de yeniden kurulmaz).
- Sadece ses ise (audio) görünür video yok; arka planda audio çalar, sadece “şu an çalışıyor” mini-bar görünür.

Player kuralı: Tek `YoutubePlayerController` / `WebViewController` / `AudioPlayer` instance kullan. `nowPlaying.videoId` değişmedikçe **asla** dispose/recreate etme. (Bu, §2.3'teki "yeniden başlamasın/kapanmasın" maddelerinin teknik karşılığıdır.)

Stream URL çözümü:

```
GET /api/chat/youtube-stream?videoId={id}
→ { success, videoId, audioUrl, videoUrl, streamUrl, youtubeUrl, title, duration,
    thumbnail, source }
```

`streamUrl` en iyi seçenektir (audio→video→youtube watch fallback). Audio mod için `audioUrl`, video mod için `videoUrl` kullan; null ise `youtubeUrl` 'e düş.

6. SENKRONİZASYON — HERKES AYNI SANİYEDE

Müzik başladığında odadaki **herkes aynı saniyeden** dinler. Sonradan giren **kaldığı saniyeden** devam eder. **Hiç kimse baştan başlatmaz.**

Mekanizma (backend gerçeği): `nowPlaying.startedAt` (ISO timestamp) backend tarafından set edilir ve SSE `dj` event'inde gelir.

Flutter sync formülü:

```
final elapsedSec = (Date-
Time.now().toUtc().difference(DateTime.parse(nowPlaying.startedAt).toUtc()).inMillise
conds / 1000.0;
// Player'ı bu pozisyonundan başlat:
player.seekTo(Duration(milliseconds: (elapsedSec * 1000).round()));
```

- Yeni `dj` event geldiğinde `videoId` **aynıysa**: seek yapma (zaten senkron), sadece queue UI güncelle.
- `videoId` **değiştiiyse**: yeni şarkıyı yükle ve `elapsedSec` 'e seek et.
- Reconnect / odaya geç giriş: ilk `dj` payload'daki `startedAt` 'a göre seek → kaldığı yerden.
- Saat farkını azaltmak için: SSE `connected` event zamanı ile cihaz saatini referans alabilirsin; basit yaklaşım `startedAt` farkı yeterlidir.

7. MİKROFON — ŞARKI BAŞLAYINCA OTOMATİK KAPAN

- Şarkı başladığında (`dj.playing == true` ve yeni `nowPlaying`): **oda sahibi hariç** herkesin mikrofону **otomatik kapanır** (Agora local audio mute).
- Kullanıcı isterse tekrar **açabilir** (manuel toggle).
- Şarkı bitince (`dj.playing == false` / `nowPlaying == null`): mikrofön **otomatik normale döner** (şarkı öncesi durumuna).

Bu tamamen **client-side** Agora kontrolüdür (`engine.muteLocalAudioStream(true/false)`). Backend mikrofön state'i tutmaz. Oda sahibi tespiti: `room.ownerId == currentUserId` .

Agora bağlantısı:

```
POST /api/agora/token Body: { channelName, role: 'host' | 'audience', uid }
{ token, uid, channelName, appId }
POST /api/chat/rooms/{roomId}/voice Body: { type: 'join' | 'leave' } // VoiceSession kaydı
```

Koltuktaki/yetkili kullanıcılar `role: 'host'` (publisher), dinleyiciler `role: 'audience'` (subscriber).

8. ODAYA GİRİŞ BİLDİRİMLERİ — WEB İLE AYNİ

Koltukların altında, role göre **modal/şerit** bildirim çıkar. Backend gerçek rolü kullanır.

Mekanizma: Presence güncellemesi sırasında backend bir sistem mesajı oluşturur ve SSE `messages` ile gelir. İçerik formatı:

- Normal: `[SYSTEM_JOIN]{ad}`
- Özel rol: `[SYSTEM_VIP_JOIN:{ENTRYTYPE}]{önekli_ad}`

`ENTRYTYPE` → etiket eşlemesi (web ile **birebir**):

entryType	Etiket	Renk
ADMIN / SUPERADMIN	👑 Site Yöneticisi	kırmızı
OWNER	🏠 Oda Sahibi	sarı
FOUNDER	~Kurucu	kırmızı
MODERATOR	&Moderatör	turuncu
OP	@Operatör	yeşil
DIAMOND	💎 Diamond Üye	camgöbeği
GOLD	🥇 Gold Üye	sarı
PREMIUM	★ Premium Üye	mor

Görünüm: “ Diamond Üye **Ahmet** odaya giriş yaptı.” / “~Kurucu **Ahmet** odaya giriş yaptı.” gibi. Üyelik (DIAMOND/GOLD/PREMIUM) için etiket+ad+“odaya giriş yaptı”, yetki rolleri için ikon+ad+“odaya giriş yaptı”.

Presence heartbeat:

```
POST /api/chat/rooms/{roomId}/presence Body: { nickname?, seatIndex? } // ~her 30-60 sn ping
```

Backend aynı kullanıcı için 5 dakikada bir tek giriş mesajı üretir (spam önleme).

9. YETKİ POPUP — KULLANICIYA TIKLAYINCA

Admin / oda sahibi / yetkililer bir kullanıcıya tıklayınca popup açılır. Aynı popup **3 yerden** açılır: **Chat mesajı, Koltuk, Katılımcı listesi**.

Popup içinde **kutular halinde** (her biri backend yetki kontrolüne tabi — yetersizse buton gizli/pasif):

Buton	Endpoint / Aksiyon
 Hediye At	POST /api/chat/rooms/{roomId}/gifts Body { giftTypeId, quantity, recipientId }
+V Ses Ver	POST .../moderation { action:'set_role', role:'voice', targetUserId }
@ Moderatör (Op)	{ action:'set_role', role:'op', targetUserId }
& Yetkili (Sop)	{ action:'set_role', role:'sop', targetUserId }
Yetki Al	{ action:'remove_role', targetUserId }
Sesi Kapat / Aç	{ action:'mute_user' / 'unmute_user', targetUserId, reason?, duration? }
Koltuğa Al / İndir	POST .../seats { targetUserId, seatIndex } (indir → seatIndex:-1)
DJ Yap / DJ'den Çıkar	POST .../dj { action:'add_dj'/'remove_dj', userId }
Odayı Devret	POST .../transfer-ownership { newOwnerId }
Mic Aç/Kapat	Agora local (kendi mic'i) / yetkili başkasını sustur → mute_user
Banla	{ action:'ban_user', targetUserId, reason?, duration? }
Sessize Al	{ action:'mute_user', targetUserId }

Tüm moderation aksiyonları: POST /api/chat/rooms/{roomId}/moderation Body { action, targetUserId, role?, reason?, duration?, message?, ttl? }.

Geçerli action değerleri:

mute_user, unmute_user, kick_user, ban_user, unban_user, mute_room, unmute_room, set_role, remove_role, clear_messages, set_owner, remove_owner, announce .

Backend yetki kontrollerini **kendisi** yapar (rol hiyerarşisi: superadmin % > founder ~ > sop & > op @ > voice +). Yetkisizse 403 döner; Flutter butonları gizlemek için GET /api/chat/rooms/{roomId}/moderation (yetkili ise rolleri/ban/mute listesini döner) veya presence'daki chatRole / roleLevel / isAdmin alanlarını kullanır.

10. ARKAPLAN — ORİJİNAL ÇÖZÜNÜRLÜK

Üstteki arkaplan butonu backend'deki oda arkaplanlarını gösterir:

```
GET /api/chat/rooms/backgrounds → { success, backgrounds:[{ roomId, slug, name, backgroundImage }] }
```

- Görseller **orijinal çözünürlükte** yüklenir. **Düşük kaliteli/sıkıştırılmış yükleme YASAK** — `Image.network` ile `cacheWidth/cacheHeight` küçük verme; orijinal URL'yi tam çözünürlükte kullan, `BoxFit.cover`.
- Arkaplanı sadece **oda sahibi / admin / yönetici** değiştirebilir:

```
PATCH /api/chat/rooms/{roomId}/settings Body: { backgroundImage: <url> }
```

(Yetki backend'de kontrol edilir; başkası için buton gizli.)

11. ODA KOMUTLARI — ÇARK SİMGESİ

Çark (⚙️) simgesine basınca **kutular halinde** komutlar açılır:

`!duyuru`, `!temizle`, `!kick`, `!ban`, `!unban`.

Her komut kutusu yetkiye göre görünür/aktif olur (op+ gerekir; `presence.roleLevel` veya `GET .../moderation` ile kontrol). Komut kutuları aşağıdaki bölümlere bağlanır.

12. !duyuru

- Tıklayınca yazı alanı açılır.

```
POST /api/chat/rooms/{roomId}/moderation Body: { action:'announce', message:<metin>, ttl?:15 }
```

- Backend SSE `system event`'i gönderir: `{ event:'ANNOUNCEMENT', text, ttl, moderator, timestamp }`.
- Flutter bu duyuruyu **chatin en üstüne sabitler, 15 saniye** gösterir, sonra otomatik siler.
- Sonradan giren** kullanıcı da görsün: backend `[ANNOUNCEMENT]{text}` olarak chat mesajı da oluşturur; yeni giren `messages` ile bunu çeker. Flutter `[ANNOUNCEMENT]` prefix'li mesajı, `timestamp + 15sn` dolmadıysa üstte sabit gösterir, dolmuşsa gizler.

13. !temizle

```
POST /api/chat/rooms/{roomId}/moderation Body: { action:'clear_messages' }
```

- Backend tüm mesajları siler + SSE `system { event:'CHAT_CLEARED', moderator }` gönderir.

- Flutter: sohbeti tamamen temizler, **komut penceresini kapatır**.
- Koltukların altında animasyon: **açık yeşil, saydam, beyaz yazılı “Sohbet Temizlendi” → 3 kez yanıp söner** (her odadaki kullanıcıda, CHAT_CLEARED event’i ile tetiklenir).

14. !kick

- Yetkiye göre kullanıcı listesi açılır. **Renk kodları:**
-  **Yeşil:** atılabilir
-  **Sarı:** aynı yetki
-  **Kırmızı:** yetki yetersiz
-  **Siyah:** Admin / Yönetici (dokunulamaz)
- Renk hesabı: hedefin `roleLevel` ‘i ile kendi `roleLevel` ‘in karşılaştırılır (presence verisinden). `isAdmin` ise siyah.

```
POST /api/chat/rooms/{roomId}/moderation Body: { action:'kick_user', targetUserId }
```

- Kick atınca odada **“Kullanıcı kicklendi”** mesajı **5 saniye** görünür (SSE `system { event:'USER_KICKED', userName, kickCount }`).
- Kicklenen kullanıcıya **uyarı** gösterilir (kickCount’a göre):

“3 ihtardan 1’ini aldınız. 2 kez daha kicklenirseniz odadan atılacaksınız. Oda sahibi banı kaldırırsa kadar giriş yapamayacaksınız.”
- Backend **3. kick’te otomatik ban** uygular (`autoBanned:true` , SSE `USER_BANNED`). Bu mantık backend’de hazır — Flutter sadece event’leri gösterir.

15. !ban

- Aynı renk sistemi (§14) geçerli. Sadece **yetkisi yeten** kişi banlanabilir (backend `403` ile korur).

```
POST /api/chat/rooms/{roomId}/moderation Body: { action:'ban_user', targetUserId, reason?, duration? }
```

(`duration` dakika; verilmezse kalıcı.) SSE `system { event:'USER_BANNED', userName, reason }`.

16. !unban

- **Banlı kullanıcı listesi** açılır:

```
GET /api/chat/rooms/{roomId}/moderation → { ..., bans:[...] } (yetkili için)
```

- İstenilen kişinin banı kaldırılır:

```
POST /api/chat/rooms/{roomId}/moderation Body: { action:'unban_user', targetUserId }
```

17. ŞARKI ARAMA POPUP — WEB İLE BİREBİR

- Yazılan **her harfte** YouTube'dan anlık arama (debounce 250ms).

```
GET /api/youtube/search?q={query}
→ { videos:[{ id, title, thumbnail, duration, channel, views } ] }
```

Alternatif (YouTube Data API v3, süre bilgisi daha doğru): `GET /api/music/search?q={query} → { items:[{ videoId, title, thumbnail, channelTitle, duration } ] }`. Web `youtube/search` kullanılır; **ikisinden hangisini web kullanıyorsa onu kullan** (varsayılan: `/api/youtube/search`).

- İlk 3 sonuç hemen** gösterilir; aşağı kaydırınca diğerleri gelir (lazy list).
- Bir müzik seçilince §3 popup'ı çıkar: **Ses 10 Jeton / Video 20 Jeton**.
- Gönderince §2.2 akışı: boşta ise hemen çalar, müzik varsa kuyruğa eklenir.

18. PERFORMANS

- YouTube araması **gecikmesiz**: debounce 250ms + sonuç **ön belleği** (query → results map, TTL ~60sn).
- Müzik oynatıcı **yeniden oluşturulmaz** (tek instance, §5).
- Kuyruk **bozulmaz**: tek kaynak `dj.musicQueue`, local mutasyon yok.
- SSE olayları optimize**: `dj event`'inde `videoId` değişmediyse player'a dokunma; sadece değişen alanları render et. `presence` 10sn'de bir gelir, gereksiz rebuild yapma (liste diff'le).
- Stream URL `youtube-stream` 60sn cache'li — aynı video için tekrar çözme.

19. KABUL KRİTERLERİ (DOĞRULAMA)

- [] Hiçbir mock veri yok — tüm veriler yukarıdaki gerçek endpoint'lerden.
- [] Web sitesiyle aynı backend (`canlifal.com`), yeni API yazılmadı.
- [] §1 butonları kaldırıldı.
- [] !istek ve popup ile müzik gönderiliyor; çalan müzik yeniden başlamıyor/kapanmıyor; kuyruk backend ile senkron.
- [] Jeton seçimi (10/20) backend fiyatından geliyor.
- [] 6 dk limiti hem ses hem video için uygulanıyor.
- [] Videolu müzik ortada floating player; chat yazarken kapanmıyor.
- [] Yeni giren kullanıcı `startedAt` 'a göre kaldığı saniyeden dinliyor; herkes senkron.
- [] Şarkı başlayınca oda sahibi hariç mic kapanıyor, bitince geri açılıyor.
- [] Giriş bildirimleri role göre koltuk altında çıkıyor (gerçek rol).
- [] Yetki popup'ı 3 yerden açılıyor; tüm aksiyonlar backend yetkisiyle çalışıyor.
- [] Arkaplan orijinal çözünürlükte; sadece sahip/admin/yönetici değiştirebiliyor.

- [] Çark komutları (!duyuru/!temizle/!kick/!ban/!unban) kutular halinde ve tam davranışıyla çalışıyor.
- [] Arama popup'ı her harfte, ilk 3 sonuç + lazy load.
- [] Acceptance testleri: **0 FAIL / 0 WARNING** → ancak o zaman APK/AAB.

EK A — SES ÇIKMAMA SORUNU (YouTube Stream Çözümü)

Web'de müzik bulunuyor ama bazen ses çıkmıyorsa kök neden **stream URL çözümü** olabilir.

Backend zinciri (/api/chat/youtube-stream):

1. Piped instance'ları (pipedapi.kavin.rocks , pipedapi.adminforge.de , piped-api.in.projectsegfault) sırayla denenir (60sn cache).
2. Hepsini başarısızsa youtubeUrl (watch linki) döner.

Flutter tarafı kuralı: Önce audioUrl (audio mod) / videoUrl (video mod) dene; yoksa streamUrl ; o da yoksa youtubeUrl (watch linki — youtube_player_flutter ile oynatılabilir). googlevideo.com URL'leri doğrudan AudioPlayer / ExoPlayer ile çalınır.

Not: Şu an backend'de ayrı bir youtube-audio proxy endpoint'i **yoktur**. Yeni endpoint **yazma** (kural gereği). Mevcut sözleşme: /api/chat/youtube-stream çözümlenmiş URL'leri döner. Eğer ileride 403/CORS sorunu için bir proxy gerekirse, bu **backend ekibiyle** ayrıca kararlaştırılmalıdır.

Eğer tüm çözümüme başarısız olursa (source: 'fallback' ve oynatma başlamazsa) kullanıcıya **sessizlik yerine görünür hata** göster:

“⚠ Şarkı oynatılamadı, lütfen başka bir şarkı deneyin.”

Not: Piped instance sağlığı / yt-dlp güncelliği gibi **sunucu tarafı** bakımlar backend ekibinin sorumluluğundadır; Flutter sadece yukarıdaki endpoint sözleşmesine uyar ve hata durumunu görünür kılar.

EK B — VERİ MODELLERİ (DTO ÖZET)

```
class NowPlaying { String videoId; String title; String? startedAt; String? duration; }
class QueueItem { String id; String videoId; String title; String dedication; String note; String duration; String requestType; bool isPaid; String userId; String userName; String createdAt; }
class DjEvent { String type='dj'; String event='QUEUE_UPDATED'; bool playing; NowPlaying? nowPlaying; String? musicUrl; List<QueueItem> musicQueue; int queueLength; }
class PresenceUser { String id; String name; String nickname; String? chatRole; String? roleSymbol; int roleLevel; bool isAdmin; }
class ChatMessage { String id; String userId; String content; String createdAt; /* [SYSTEM_JOIN], [SYSTEM_VIP_JOIN:TYPE], [ANNOUNCEMENT], [SONG_REQUEST_*] prefixleri parse edilir */ }
class SystemEvent { String event; String? text; int? ttl; String? userName; int? kickCount; String? reason; String? moderator; }
```

Özet: Bu döküman Flutter sesli sohbet ekranını, web'in kullandığı **mevcut** backend sözleşmesine birbir oturtur. Yeni endpoint yoktur; tüm state SSE `dj / messages / presence / system / gift` event'lerinden ve listelenen REST endpoint'lerinden gelir.